DATA 260: Agentic AI

Instructor: Prof. Dr. Simon Shim | Student Name: Trapti Kulshrestha | STUDENT ID: 019124833 | Homework: 8

GitHub Repository: https://github.com/Psycoder0611/data260-4833

Section 0: Personal Configuration

Setting	Value
SID4	4833
PORT_BASE	8333 (8000 + 4833 mod 900)
PREFIX	s4833
SEED	4833
VERIFY_SEED	264833
DOMAIN_ID	1 (4833 mod 8)
Assigned Domain	Clinical Trial Listings

Hardware: Apple MacBook Pro

Local Model: qwen3:8b (Ollama), seed 4833. No hosted API is used.

Message broker: Apache Kafka 3.9.2 (KRaft mode) in Docker; Python client aiokafka 0.14.0

Project Setup

For Homework 8 I kept working in my existing DATA 260 repository, on a hw8-work branch made from the hw7 tag. All new code is in the hw8/ package and a few top-level scripts, so none of the earlier homework is changed. Kafka runs from docker-compose.hw8.yml (one broker, KRaft, so no ZooKeeper). A small init container creates the seven topics with the partition counts I need and automatic topic creation is switched off, so a typo in a topic name fails loudly instead of creating a new topic. The worker and the FastAPI wrapper are two more containers built from the same image. The Planner, Writer and Reviewer for Part 2 run on my Mac, outside Docker, because they call Ollama on the host.

Topics (all prefixed s4833): entity_requests, entity_responses, inbox, drafts, final and dead_letter have one partition each; tasks has two partitions, because Part 3 needs two Writers to share it.

Part 1 - Kafka Request/Response

1.1 Message format and validation

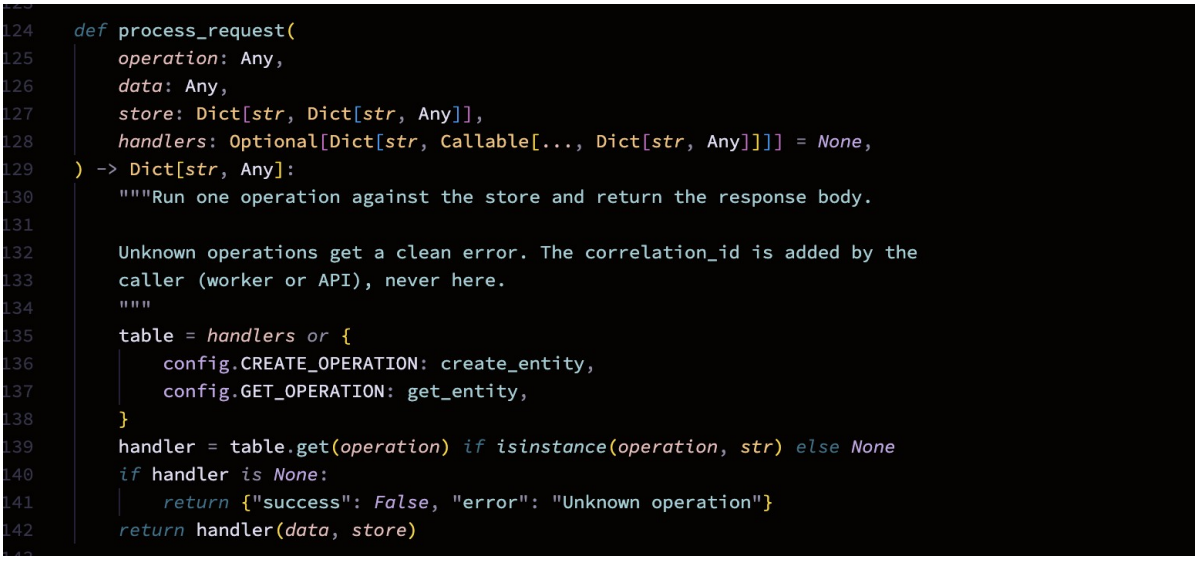
A client sends a request to s4833_entity_requests and waits for the answer on s4833_entity_responses. A request has four fields: correlation_id (a UUID made by the client), reply_to (the topic the answer must go to), operation and data. The worker supports two operations for my domain, CREATE_CLINICAL_TRIAL and GET_CLINICAL_TRIAL. A create needs trialTitle, sponsor, trial_code, participant_capacity (an integer of at least 1), submitterEmail and category; description and organization_id are optional.

The worker never crashes on bad input. If the envelope is wrong (no correlation_id, no reply_to, data that is not an object, a message that is not JSON) it logs it and, when it can still find a reply topic, answers with success false and an error. If the operation is unknown, the data is invalid or the trial id does not exist, the answer is still a normal response with success false, a short error text and the same correlation_id, so the client can match the error to its request. The two screenshots below show the validation code and process_request, which turns every request into a response.



```python
def validate_create(data: Any) -> List[str]:
    """Return a list of problems with a create payload (empty list = valid)."""
    if not isinstance(data, dict):
        return ["data must be an object"]
    problems: List[str] = []
    for name in REQUIRED_FIELDS:
        if name not in data or data[name] is None:
            problems.append(f"{name} is required")
    for name, (low, high) in TEXT_FIELDS.items():
        value = data.get(name)
        if value is None:
            continue
        if not isinstance(value, str):
            problems.append(f"{name} must be a string")
        elif not value.strip():
            problems.append(f"{name} must not be empty")
        elif not low <= len(value.strip()) <= high:
            problems.append(f"{name} must be {low} to {high} characters")
    capacity = data.get("participant_capacity")
    if capacity is not None:
        if isinstance(capacity, bool) or not isinstance(capacity, int):
            problems.append("participant_capacity must be an integer")
        elif capacity < 1:
            problems.append("participant_capacity must be at least 1")
    email = data.get("submitterEmail")
    if email is not None:
        if not isinstance(email, str) or not EMAIL_PATTERN.match(email.strip()):
            problems.append("submitterEmail must be a valid email address")
    description = data.get("description")
    if description is not None and (
        not isinstance(description, str) or len(description) > 2000
```

Figure 1 - Code: validate_create (hw8/entity_service.py)



```python
def process_request(
    operation: Any,
    data: Any,
    store: Dict[str, Dict[str, Any]],
    handlers: Optional[Dict[str, Callable[..., Dict[str, Any]]]] = None,
) -> Dict[str, Any]:
    """Run one operation against the store and return the response body.

    Unknown operations get a clean error. The correlation_id is added by the
    caller (worker or API), never here.
    """
    table = handlers or {
        config.CREATE_OPERATION: create_entity,
        config.GET_OPERATION: get_entity,
    }
    handler = table.get(operation) if isinstance(operation, str) else None
    if handler is None:
        return {"success": False, "error": "Unknown operation"}
    return handler(data, store)
```

Figure 2 - Code: process_request (hw8/entity_service.py)

1.2 The worker

The worker joins the consumer group s4833_entity_worker and logs 'Consumer is ready and listening...' once partitions are assigned. For every request it logs REQUEST RECEIVED, PROCESSING, PROCESSED and RESPONSE SENT with the correlation_id, so one request can be followed through the log with a search. It commits the Kafka offset only after the response has been sent. If the same correlation_id arrives a second time, the worker sends back the stored first response instead of creating a second trial. Figures 4 and 5 show the containers and the ready message. Figure 5 was taken after restarting the worker container: the last lines of the log are a few earlier requests, then the ready line with the request and response topics.

```python
    async def handle(self, record: Any) -> None:
        """Process one request record and commit its offset after the reply."""
        where = (
            f"topic={record.topic} partition={record.partition} offset={record.offset}"
        )
        raw_text = (record.value or b"").decode("utf-8", "replace")
        correlation_id: Optional[str] = None
        reply_to = config.RESPONSE_TOPIC
        operation: Any = None
        try:
            request = kafka_io.loads(record.value)
        except ValueError as exc:
            self.log.info("REQUEST RECEIVED %s (unreadable)", where)
            self.trace("received", where=where, raw=raw_text[:200])
            result: Dict[str, Any] = {"success": False, "error": f"Bad message: {exc}"}
        else:
            raw_cid = request.get("correlation_id")
            correlation_id = raw_cid if isinstance(raw_cid, str) and raw_cid else None
            operation = request.get("operation")
            candidate = request.get("reply_to")
            if isinstance(candidate, str) and candidate.startswith(f"{config.PREFIX}_"):
                reply_to = candidate
            self.log.info(
                "REQUEST RECEIVED %s correlation_id=%s", where, correlation_id
            )
            self.log.info("REQUEST %s", json.dumps(request))
            self.trace(
                "received",
                correlation_id=correlation_id,
                operation=operation,
                partition=record.partition,
```

Figure 3 - Code: worker handle (hw8/entity_worker.py)

```
(base) Triptis-MacBook-Pro ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(1d10h33m|git@hw8-work!)
2118 ± docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS                    PORTS                      N
83d6604abe3a   s4833-hw8            "python -m hw8.entit…"   16 seconds ago   Up 2 seconds                                         s
5ef50297920d   s4833-hw8            "uvicorn hw8.api:app…"   16 seconds ago   Up 2 seconds              127.0.0.1:8333->8333/tcp   s
439f06c637ea   apache/kafka:3.9.2   "/__cacert_entrypoin…"   17 seconds ago   Up 16 seconds (healthy)   127.0.0.1:9092->9092/tcp   s
d46c431a8cf6   mongo:7.0            "docker-entrypoint.s…"   10 days ago      Up 15 minutes (healthy)   127.0.0.1:27017->27017/tcp h
```

Figure 4 - docker ps: Kafka, worker and API containers running

```
2161 ± docker compose -f docker-compose.hw8.yml restart worker
sleep 10
docker compose -f docker-compose.hw8.yml logs --tail 6 worker
[+] restart 0/1
 ⠼ Container s4833-worker Restarting                                                
```

```python
@app.post("/api/hw8/entity")
async def entity_over_kafka(
    body: EntityRequest,
    x_correlation_id: Optional[str] = Header(default=None),
) -> JSONResponse:
    """Send the operation to the Kafka worker and return its response."""
    correlation_id = x_correlation_id or str(uuid.uuid4())
    try:
        response = await rpc.request(body.operation, body.data, correlation_id)
    except RpcTimeout as exc:
        log.info("TIMEOUT correlation_id=%s", exc.correlation_id)
        return JSONResponse(
            status_code=504,
            content={
                "correlation_id": exc.correlation_id,
                "success": False,
                "error": "Timeout waiting for the Kafka worker",
            },
        )
    if config.API_VERBOSE:
        log.info(
            "RETURNING HTTP response to caller correlation_id=%s: %s",
            correlation_id,
            json.dumps(response),
        )
    return JSONResponse(content=response)

```

Figure 6 - Code: POST /api/hw8/entity over Kafka (hw8/api.py)

```python

@app.post("/api/hw8/entity-direct")
async def entity_direct(
    body: EntityRequest,
    x_correlation_id: Optional[str] = Header(default=None),
) -> JSONResponse:
    """Run the same operation in this process (no Kafka) for the JMeter baseline."""
    correlation_id = x_correlation_id or str(uuid.uuid4())
    result = process_request(body.operation, body.data, direct_store)
    response = {"correlation_id": correlation_id, "operation": body.operation, **result}
    if config.API_VERBOSE:
        log.info(
            "DIRECT (no Kafka) correlation_id=%s: %s",
            correlation_id,
            json.dumps(response),
        )
    return JSONResponse(content=response)

```

Figure 7 - Code: /api/hw8/entity-direct baseline (hw8/api.py)

```
(base) Triptis-MacBook-Pro 🐼 ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(1d10h33m|git@hw8-work!)
2120 ± docker compose -f docker-compose.hw8.yml logs api
s4833-api  | INFO:     Started server process [1]
s4833-api  | INFO:     Waiting for application startup.
s4833-api  | 04:43:07.070 [FASTAPI] Listening for responses on topic: s4833_entity_responses
s4833-api  | 04:43:07.070 [FASTAPI] FastAPI server started at http://127.0.0.1:8333
s4833-api  | 04:43:07.070 [FASTAPI] Kafka topics: requests=s4833_entity_requests responses=s4833_entity_responses
s4833-api  | INFO:     Application startup complete.
s4833-api  | INFO:     Uvicorn running on http://0.0.0.0:8333 (Press CTRL+C to quit)
```

Figure 8 - FastAPI log: started at http://127.0.0.1:8333

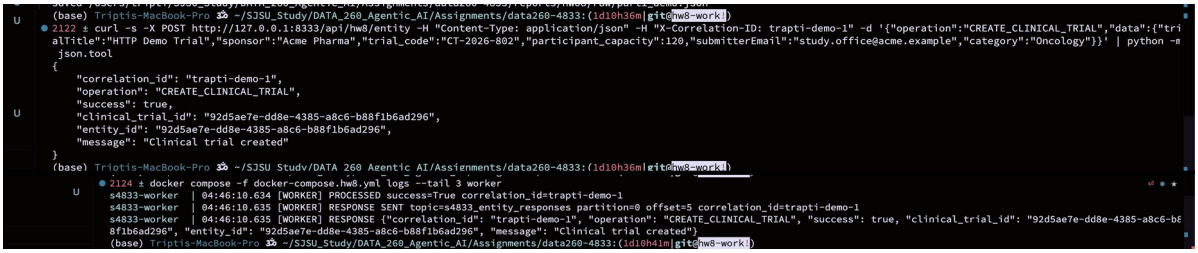


Figure 9 - POST /api/hw8/entity through Kafka returns the response

1.4 homework_demo.py

homework_demo.py shows the ten steps the assignment lists: client starts, builds the request, generates the UUID, publishes, the worker receives it, validates, processes, sends the response, the client matches the correlation_id and prints the result. The worker's own log lines are merged into the same output so the whole round trip reads in order. Figure 12 is the success path. After the ten steps it also sends an unknown operation, invalid data and an unknown id; each returns success false with the original correlation_id (Figure 13). Figure 9 shows the same round trip over HTTP. The two code screenshots above them are the demo steps and the Kafka client (rpc_client.py) that sends the request and waits for the matching correlation_id.

```python
async def demo(show_errors: bool) -> int:
    """Run the full demonstration; return a process exit code."""
    rpc = KafkaRPC(component="CLIENT", verbose=True, timeout=15)
    await rpc.start()
    summary: Dict[str, Any] = {"topics": [config.REQUEST_TOPIC, config.RESPONSE_TOPIC]}
    try:
        print("\n=== STEP 1: create one correlation_id for the CREATE request ===")
        create_cid = str(uuid.uuid4())
        log.info("correlation_id for CREATE = %s", create_cid)

        print("\n=== STEPS 2-6: CREATE request -> worker -> response ===")
        created = await call(rpc, config.CREATE_OPERATION, TRIAL, create_cid)
        print("\n  CREATE response:")
        print(json.dumps(created, indent=2))
        assert created["correlation_id"] == create_cid, "correlation_id changed!"
        if not created.get("success"):
            log.info("CREATE failed, stopping")
            return 1

        print("\n=== STEP 7: extract the generated id ===")
        entity_id = created[config.ID_FIELD]
        log.info("%s = %s", config.ID_FIELD, entity_id)

        print("\n=== STEPS 8-9: GET request with that id (its own correlation_id) ===")
        get_cid = str(uuid.uuid4())
        log.info("correlation_id for GET = %s", get_cid)
        got = await call(
            rpc, config.GET_OPERATION, {config.ID_FIELD: entity_id}, get_cid
        )

        print("\n=== STEP 10: retrieved entity and correlation_id check ===")
        print(json.dumps(got, indent=2))
        create_ok = created["correlation_id"] == create_cid
        get_ok = got["correlation_id"] == get_cid
        log.info(
            "CREATE correlation_id preserved: sent=%s received=%s -> %s",
            create_cid, created["correlation_id"], "YES" if create_ok else "NO",
        )  # fmt: skip
        log.info(
            "GET    correlation_id preserved: sent=%s received=%s -> %s",
            get_cid, got["correlation_id"], "YES" if get_ok else "NO",
        )  # fmt: skip
        same = {k: v for k, v in got.get("entity", {}).items() if k in TRIAL} == TRIAL
        log.info(
```

Figure 10 - Code: homework_demo.py, demo() steps 1-10

```python
    async def request(
        self,
        operation: str,
        data: Dict[str, Any],
        correlation_id: Optional[str] = None,
    ) -> Dict[str, Any]:
        """Send one request and return the full response message.

        The correlation_id is created here once (unless the caller supplies
        one) and is never changed afterwards.
        """
        correlation_id = correlation_id or str(uuid.uuid4())
        message = {
            "correlation_id": correlation_id,
            "reply_to": self.response_topic,
            "operation": operation,
            "data": data,
        }
        future: asyncio.Future = asyncio.get_running_loop().create_future()
        self.pending[correlation_id] = future
        metadata = await self.producer.send_and_wait(
            self.request_topic, kafka_io.dumps(message)
        )
        self.events.append(
            {
                "ts": time.time(),
                "component": "CLIENT",
                "event": "request_sent",
                "correlation_id": correlation_id,
                "operation": operation,
                "topic": metadata.topic,
                "partition": metadata.partition,
                "offset": metadata.offset,
```

Figure 11 - Code: Kafka client request/wait (hw8/rpc_client.py)



Figure 12 - homework_demo.py: the ten logged steps, success path



Figure 13 - homework_demo.py: unknown operation, bad data and unknown id

Part 2 - Planner, Writer and Reviewer over Kafka

The three agents are separate Python processes (planner.py, writer.py, reviewer.py). Each one reads from one topic and writes to the next: a question goes into s4833_inbox, the Planner writes a plan to s4833_tasks, the Writer writes a draft to s4833_drafts, and the Reviewer scores it and writes the result to s4833_final. The correlation_id is created once in send_question.py and copied unchanged by every agent, so one grep follows the whole journey. The Planner, Writer and Reviewer call qwen3:8b through the HW1 model client with seed 4833; the calls run in a worker thread so the Kafka heartbeat keeps going while the model is thinking. Figure 17 shows the three processes ready.

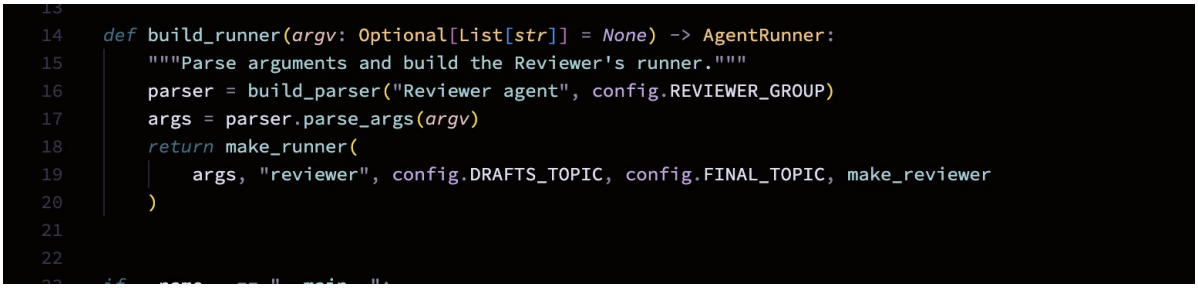
send_question.py --wait --journey sends one question and then reads all four topics to print every hop with its partition and offset (Figure 18). The final message holds the status (approved or rejected, from a review score of at least 6 out of 10) and the answer (Figure 19).

```python
13
14  def build_runner(argv: Optional[List[str]] = None) -> AgentRunner:
15      """Parse arguments and build the Planner's runner."""
16      parser = build_parser("Planner agent", config.PLANNER_GROUP)
17      args = parser.parse_args(argv)
18      return make_runner(
19          args, "planner", config.INBOX_TOPIC, config.TASKS_TOPIC, make_planner
20      )
21
22
```

Figure 14 - Code: Planner (planner.py)

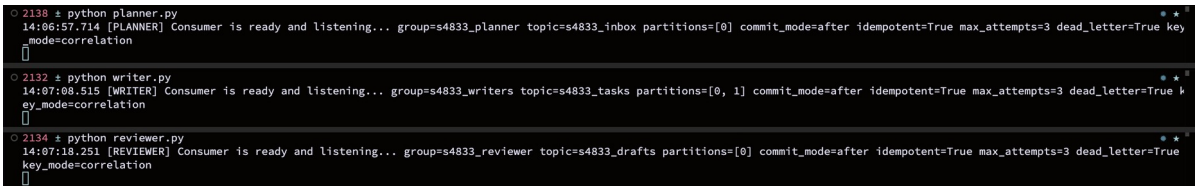
```python
17  def build_runner(argv: Optional[List[str]] = None) -> AgentRunner:
18      """Parse arguments and build the Writer's runner."""
19      parser = build_parser("Writer agent", config.WRITER_GROUP)
20      parser.add_argument(
21          "--record-path",
22          default=None,
23          help="append one line per finished message here (ordering test)",
24      )
25      args = parser.parse_args(argv)
26      record_path = Path(args.record_path) if args.record_path else None
```

Figure 15 - Code: Writer (writer.py)



```
def build_runner(argv: Optional[List[str]] = None) -> AgentRunner:
    """Parse arguments and build the Reviewer's runner."""
    parser = build_parser("Reviewer agent", config.REVIEWER_GROUP)
    args = parser.parse_args(argv)
    return make_runner(
        args, "reviewer", config.DRAFTS_TOPIC, config.FINAL_TOPIC, make_reviewer
    )
```

Figure 16 - Code: Reviewer (reviewer.py)

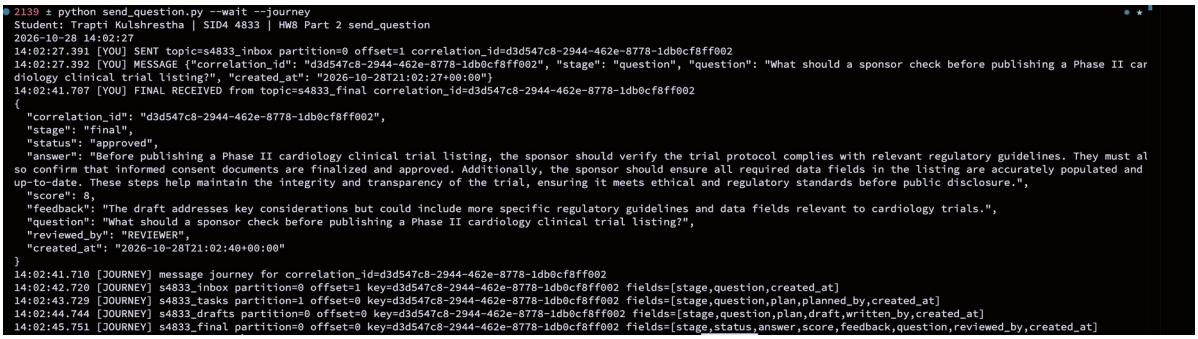


```
2138 ± python planner.py
14:06:57.714 [PLANNER] Consumer is ready and listening... group=s4833_planner topic=s4833_inbox partitions=[0] commit_mode=after idempotent=True max_attempts=3 dead_letter=True key_mode=correlation

2132 ± python writer.py
14:07:08.515 [WRITER] Consumer is ready and listening... group=s4833_writers topic=s4833_tasks partitions=[0, 1] commit_mode=after idempotent=True max_attempts=3 dead_letter=True key_mode=correlation

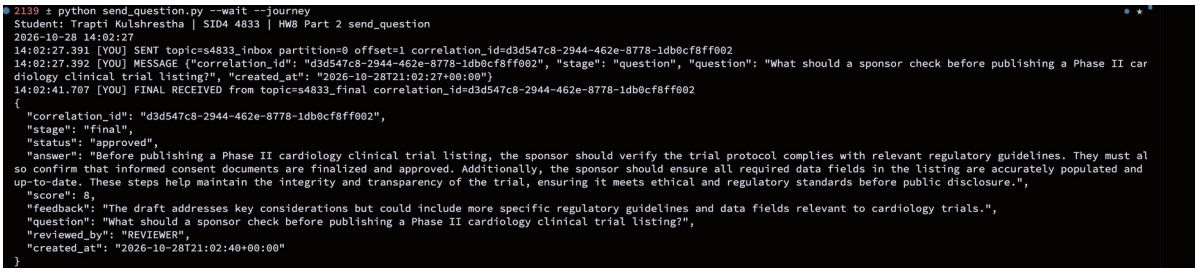
2134 ± python reviewer.py
14:07:18.251 [REVIEWER] Consumer is ready and listening... group=s4833_reviewer topic=s4833_drafts partitions=[0] commit_mode=after idempotent=True max_attempts=3 dead_letter=True key_mode=correlation
```

Figure 17 - Planner, Writer and Reviewer running as three processes



```
2139 ± python send_question.py --wait --journey
Student: Trapti Kulshrestha | SID4 4833 | HW8 Part 2 send_question
2026-10-28 14:02:27
14:02:27.391 [YOU] SENT topic=s4833_inbox partition=0 offset=1 correlation_id=d3d547c8-2944-462e-8778-1db0cf8ff002
14:02:27.392 [YOU] MESSAGE {"correlation_id": "d3d547c8-2944-462e-8778-1db0cf8ff002", "stage": "question", "question": "What should a sponsor check before publishing a Phase II cardiology clinical trial listing?", "created_at": "2026-10-28T21:02:27+00:00"}
14:02:41.707 [YOU] FINAL RECEIVED from topic=s4833_final correlation_id=d3d547c8-2944-462e-8778-1db0cf8ff002
{
  "correlation_id": "d3d547c8-2944-462e-8778-1db0cf8ff002",
  "stage": "final",
  "status": "approved",
  "answer": "Before publishing a Phase II cardiology clinical trial listing, the sponsor should verify the trial protocol complies with relevant regulatory guidelines. They must also confirm that informed consent documents are finalized and approved. Additionally, the sponsor should ensure all required data fields in the listing are accurately populated and up-to-date. These steps help maintain the integrity and transparency of the trial, ensuring it meets ethical and regulatory standards before public disclosure.",
  "score": 8,
  "feedback": "The draft addresses key considerations but could include more specific regulatory guidelines and data fields relevant to cardiology trials.",
  "question": "What should a sponsor check before publishing a Phase II cardiology clinical trial listing?",
  "reviewed_by": "REVIEWER",
  "created_at": "2026-10-28T21:02:40+00:00"
}
14:02:41.710 [JOURNEY] message journey for correlation_id=d3d547c8-2944-462e-8778-1db0cf8ff002
14:02:42.720 [JOURNEY] s4833_inbox partition=0 offset=1 key=d3d547c8-2944-462e-8778-1db0cf8ff002 fields=[stage,question,created_at]
14:02:43.729 [JOURNEY] s4833_tasks partition=1 offset=0 key=d3d547c8-2944-462e-8778-1db0cf8ff002 fields=[stage,question,plan,planned_by,created_at]
14:02:44.744 [JOURNEY] s4833_drafts partition=0 offset=0 key=d3d547c8-2944-462e-8778-1db0cf8ff002 fields=[stage,question,plan,draft,written_by,created_at]
14:02:45.751 [JOURNEY] s4833_final partition=0 offset=0 key=d3d547c8-2944-462e-8778-1db0cf8ff002 fields=[stage,status,answer,score,feedback,question,reviewed_by,created_at]
```

Figure 18 - One question travelling inbox -> tasks -> drafts -> final



```
2139 ± python send_question.py --wait --journey
Student: Trapti Kulshrestha | SID4 4833 | HW8 Part 2 send_question
2026-10-28 14:02:27
14:02:27.391 [YOU] SENT topic=s4833_inbox partition=0 offset=1 correlation_id=d3d547c8-2944-462e-8778-1db0cf8ff002
14:02:27.392 [YOU] MESSAGE {"correlation_id": "d3d547c8-2944-462e-8778-1db0cf8ff002", "stage": "question", "question": "What should a sponsor check before publishing a Phase II cardiology clinical trial listing?", "created_at": "2026-10-28T21:02:27+00:00"}
14:02:41.707 [YOU] FINAL RECEIVED from topic=s4833_final correlation_id=d3d547c8-2944-462e-8778-1db0cf8ff002
{
  "correlation_id": "d3d547c8-2944-462e-8778-1db0cf8ff002",
  "stage": "final",
  "status": "approved",
  "answer": "Before publishing a Phase II cardiology clinical trial listing, the sponsor should verify the trial protocol complies with relevant regulatory guidelines. They must also confirm that informed consent documents are finalized and approved. Additionally, the sponsor should ensure all required data fields in the listing are accurately populated and up-to-date. These steps help maintain the integrity and transparency of the trial, ensuring it meets ethical and regulatory standards before public disclosure.",
  "score": 8,
  "feedback": "The draft addresses key considerations but could include more specific regulatory guidelines and data fields relevant to cardiology trials.",
  "question": "What should a sponsor check before publishing a Phase II cardiology clinical trial listing?",
  "reviewed_by": "REVIEWER",
  "created_at": "2026-10-28T21:02:40+00:00"
}
```

Figure 19 - Final JSON with status and answer

Part 3 - Failure Handling

For each failure I first ran the agent with the protection switched off to see the problem, then switched the protection on and ran the same test again. The switches are command-line options on the same Writer code (--no-idempotency, --commit-mode on_receive, --max-attempts 0 --no-dead-letter), so the before and after runs differ only by the fix. Everything is started by python run_hw08_part3.py, and the Writer in these runs uses the fixed-text stand-in instead of qwen3:8b (see Self-check).

3.1 Duplicate delivery and idempotency

Kafka gives at-least-once delivery, so the same message can arrive twice. I sent one task message and then sent the same message again with the same correlation_id. Without a check the Writer did the work twice

and published two drafts (Figure 21). The fix is a small SQLite table of (stage, correlation_id) pairs that are already done. The Writer checks it before working and records the id after it has published the draft; a replay is logged as DUPLICATE, skipped, and its offset is committed so it does not come back (Figure 22).

```python
    def is_done(self, stage: str, correlation_id: str) -> bool:
        """True if this stage already finished this correlation_id."""
        row = self.connection.execute(
            "SELECT 1 FROM done WHERE stage = ? AND correlation_id = ?",
            (stage, correlation_id),
        ).fetchone()
        return row is not None

    def mark_done(self, stage: str, correlation_id: str) -> None:
        """Record that the stage finished (a second call changes nothing)."""
        self.connection.execute(
            "INSERT OR IGNORE INTO done (stage, correlation_id, first_done_at) "
            "VALUES (?, ?, ?)",
            (stage, correlation_id, time.time()),
        )

    def note_duplicate(self, stage: str, correlation_id: str) -> int:
        """Count one skipped duplicate and return the new total."""
```

Figure 20 - Code: idempotency store (hw8/idempotency.py)

```
2141 ± python run_hw08_part3.py duplicate
Student: Trapti Kulshrestha | SID4 4833 | HW8 Part 3 duplicate
2026-10-28 14:08:57
topics ready (partitions): {'s4833_inbox': 1, 's4833_final': 1, 's4833_tasks': 2, 's4833_entity_responses': 1, 's4833_entity_requests': 1, 's4833_drafts': 1, 's4833_dead_letter': 1
}

--- Duplicate delivery - BEFORE the fix ---
  $ python writer.py --instance A --group s4833_t3_dup_before_1028140857 --offset-reset latest --llm stub --state-db /Users/tripti/SJSU_Study/DATA_260_Agentic_AI/Assignments/data26
0-4833/reports/hw08/state/dup_before_1028140857.db --no-idempotency
    | 14:08:57.254 [WRITER-A] Consumer is ready and listening... group=s4833_t3_dup_before_1028140857 topic=s4833_tasks partitions=[0, 1] commit_mode=after idempotent=False max_att
empts=3 dead_letter=True key_mode=correlation
14:08:57.262 [TEST] original sent -> topic=s4833_tasks partition=0 offset=0 correlation_id=dup-before-1028140857
    | 14:08:57.262 [WRITER-A] RECEIVED topic=s4833_tasks partition=0 offset=0 correlation_id=dup-before-1028140857
    | 14:08:57.262 [WRITER-A] PROCESSING correlation_id=dup-before-1028140857 attempt=1/3
    | 14:08:57.265 [WRITER-A] SENT topic=s4833_drafts partition=0 offset=1 correlation_id=dup-before-1028140857
    | 14:08:57.267 [WRITER-A] COMMITTED offset=1 partition=0
14:08:57.320 [TEST] REPLAY of the same message sent -> topic=s4833_tasks partition=0 offset=1 correlation_id=dup-before-1028140857
    | 14:08:57.319 [WRITER-A] RECEIVED topic=s4833_tasks partition=0 offset=1 correlation_id=dup-before-1028140857
    | 14:08:57.320 [WRITER-A] PROCESSING correlation_id=dup-before-1028140857 attempt=1/3
    | 14:08:57.321 [WRITER-A] SENT topic=s4833_drafts partition=0 offset=2 correlation_id=dup-before-1028140857
    | 14:08:57.322 [WRITER-A] COMMITTED offset=2 partition=0
14:09:00.382 [TEST] RESULT before: messages sent=2, drafts produced=2, duplicates skipped=0
    | 14:09:00.386 [WRITER-A] stopped (processed 2 messages)
```

Figure 21 - Duplicate delivery without idempotency: two drafts

```
--- Duplicate delivery - AFTER the fix ---
  $ python writer.py --instance A --group s4833_t3_dup_after_1028140857 --offset-reset latest --llm stub --state-db /Users/tripti/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260
-4833/reports/hw08/state/dup_after_1028140857.db
    | 14:09:00.721 [WRITER-A] Consumer is ready and listening... group=s4833_t3_dup_after_1028140857 topic=s4833_tasks partitions=[0, 1] commit_mode=after idempotent=True max_attem
pts=3 dead_letter=True key_mode=correlation
14:09:00.782 [TEST] original sent -> topic=s4833_tasks partition=1 offset=1 correlation_id=dup-after-1028140857
    | 14:09:00.782 [WRITER-A] RECEIVED topic=s4833_tasks partition=1 offset=1 correlation_id=dup-after-1028140857
    | 14:09:00.782 [WRITER-A] PROCESSING correlation_id=dup-after-1028140857 attempt=1/3
    | 14:09:00.786 [WRITER-A] SENT topic=s4833_drafts partition=0 offset=3 correlation_id=dup-after-1028140857
    | 14:09:00.789 [WRITER-A] COMMITTED offset=2 partition=1
14:09:00.842 [TEST] REPLAY of the same message sent -> topic=s4833_tasks partition=1 offset=2 correlation_id=dup-after-1028140857
    | 14:09:00.842 [WRITER-A] RECEIVED topic=s4833_tasks partition=1 offset=2 correlation_id=dup-after-1028140857
    | 14:09:00.843 [WRITER-A] DUPLICATE correlation_id=dup-after-1028140857 already processed -> skipped, no new output (duplicates skipped so far: 1)
    | 14:09:00.845 [WRITER-A] COMMITTED offset=3 partition=1
14:09:03.911 [TEST] RESULT after: messages sent=2, drafts produced=1, duplicates skipped=1
    | 14:09:03.930 [WRITER-A] stopped (processed 1 messages)

saved reports/hw08/raw/part3_duplicate.json
(.venv) (base) Triptis-MacBook-Pro ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(2d2h59m|gite[hw8-work])
2144 ±
```

Figure 22 - Duplicate delivery with idempotency: one draft, replay skipped

3.2 Consumer killed in the middle of work

I gave the victim message a long processing time, killed the Writer with SIGKILL while it was working, and started a new Writer in the same group. If the offset is committed when the message is received, the new Writer starts after it and the message is lost: no draft is ever produced (Figure 24). With commit-after-success the offset is only committed once the draft has been published, so the dead Writer never committed it, the new Writer receives it again, finishes it and then commits (Figure 25). The cost is that a message can be processed twice if the Writer dies after publishing but before committing, which is why Part 3.1 and this part belong together.

```python
    async def process_record(self, record: Any) -> None:
        """Handle one Kafka record according to the options."""
        opts = self.options
        where = (
            f"topic={record.topic} partition={record.partition} offset={record.offset}"
        )
        try:
            payload: Optional[Dict[str, Any]] = kafka_io.loads(record.value)
        except ValueError:
            payload = None
        cid = payload.get("correlation_id") if payload else None
        cid = cid if isinstance(cid, str) and cid else None
        self.log.info("RECEIVED %s correlation_id=%s", where, cid)

        if opts.commit_mode == "on_receive":
            await self._commit(record, "(before processing)")

        if opts.idempotent and cid and self.store.is_done(opts.name, cid):
            count = self.store.note_duplicate(opts.name, cid)
            self.log.info(
                "DUPLICATE correlation_id=%s already processed -> skipped, no new "
                "output (duplicates skipped so far: %d)",
                cid,
                count,
            )
            if opts.commit_mode == "after":
                await self._commit(record)
            return

        ctx = {
            "agent": self.label,
            "instance": opts.instance,
            "partition": record.partition
```

Figure 23 - Code: process_record, commit and duplicate check (hw8/agent_base.py)

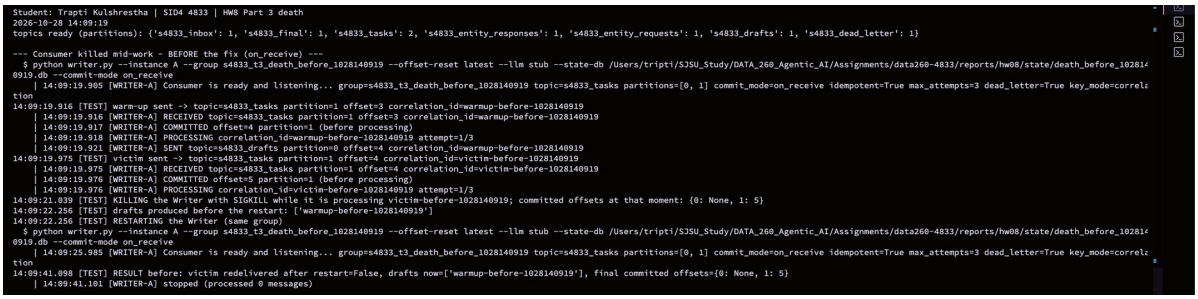


Figure 24 - Consumer killed, commit on receive: message lost

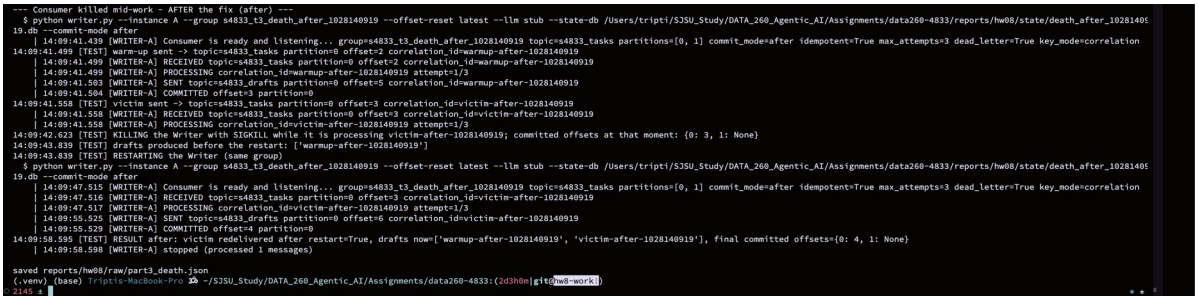


Figure 25 - Consumer killed, commit after success: message redelivered

3.3 Poison message and the dead-letter topic

A poison message is one that can never succeed. Mine has a plan field that is a number instead of a list, so the Writer raises an error every time. With unlimited retries the Writer keeps failing on the same offset and the good message behind it is never reached (Figure 27). With the fix the Writer tries three times with a growing wait, then publishes the original message, the error, the number of attempts and its source topic, partition and offset to s4833_dead_letter, commits the offset and moves on. The good message after it is processed normally (Figure 28).

```
216     async def _attempts(
217         self,
218         record: Any,
219         payload: Optional[Dict[str, Any]],
220         cid: Optional[str],
221         ctx: Dict[str, Any],
222     ) -> Any:
223         """Run the handler with retries; return its output or _DEAD_LETTERED."""
224         opts = self.options
225         limit = opts.max_attempts
226         attempt = 0
227         while True:
228             attempt += 1
229             shown = "unbounded" if limit == 0 else limit
230             try:
231                 if payload is None:
232                     raise ValueError("message value is not a valid JSON object")
233                 self.log.info(
234                     "PROCESSING correlation_id=%s attempt=%d/%s", cid, attempt, shown
235                 )
236                 return await self.handler(payload, ctx)
237             except asyncio.CancelledError:
238                 raise
239             except Exception as exc:
240                 self.log.info(
241                     "FAILED correlation_id=%s attempt=%d/%s error=%s: %s",
242                     cid,
243                     attempt,
244                     shown,
245                     type(exc).__name__,
246                     exc,
247                 )
248                 if limit and attempt >= limit:
```

Figure 26 - Code: bounded retries in _attempts (hw8/agent_base.py)

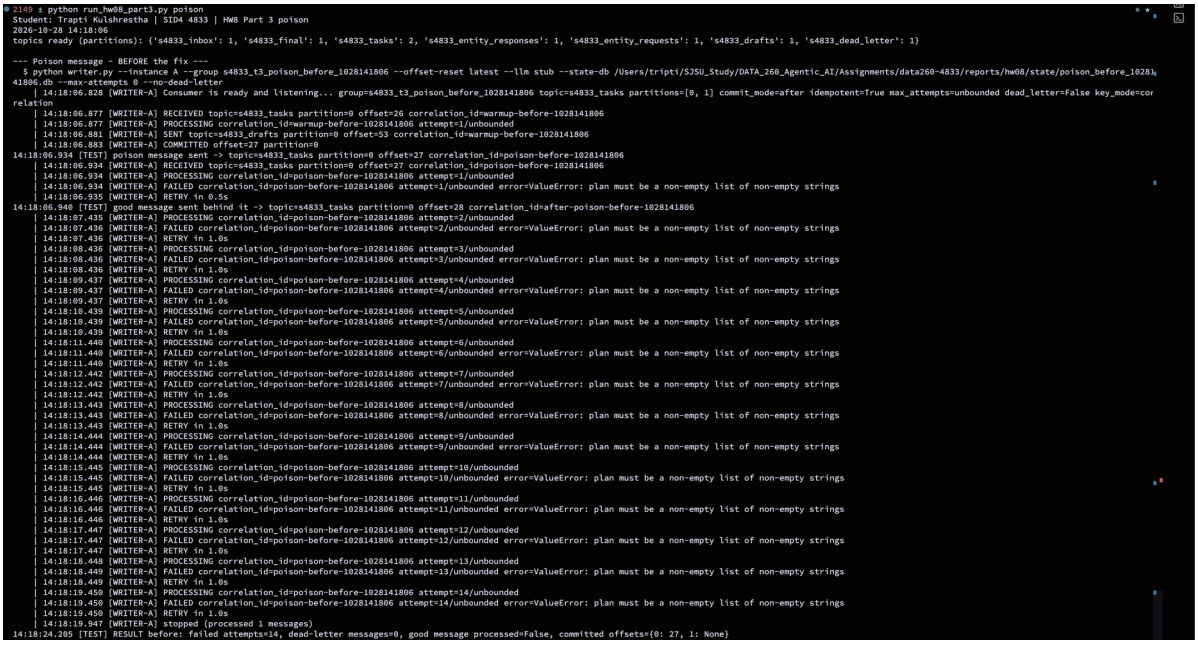


Figure 27 - Poison message without a limit: endless retries, queue blocked

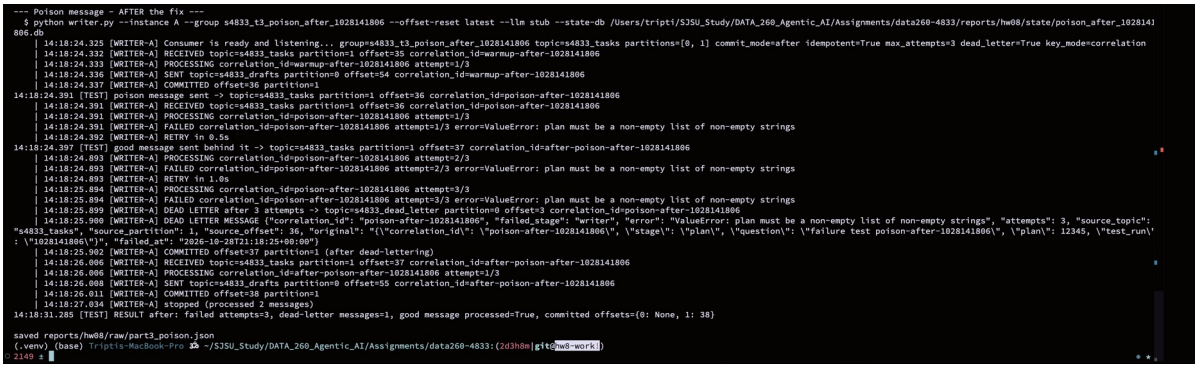
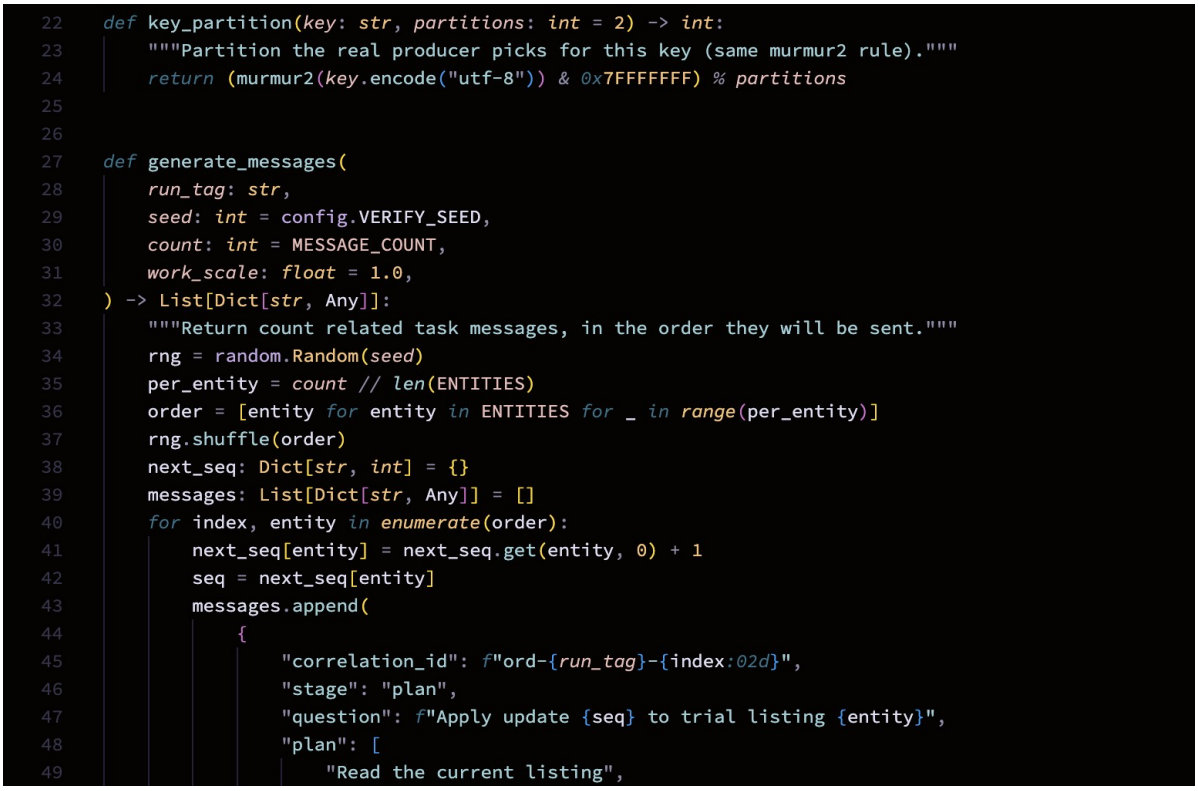


Figure 28 - Poison message with 3 attempts and the dead-letter topic

3.4 Ordering

I generated 20 messages with VERIFY_SEED 264833: five clinical-trial entities with four messages each, where the order of the four matters. s4833_tasks has two partitions and two Writers run in the same consumer group, so each Writer owns one partition. First I sent the messages with no key; Kafka then spreads them over both partitions, the two Writers work at different speeds, so the order inside one entity is not guaranteed (Figure 30). In my run 2 of 5 entities were processed out of order (2 inversions in total) and 4 entities had messages in both partitions. Then I sent the same kind of messages with the entity id as the key. The key decides the partition, so all messages of one entity go to the same partition, and one partition is read by one Writer at a time, in order (Figure 31).



```python
def key_partition(key: str, partitions: int = 2) -> int:
    """Partition the real producer picks for this key (same murmur2 rule)."""
    return (murmur2(key.encode("utf-8")) & 0x7FFFFFFF) % partitions


def generate_messages(
    run_tag: str,
    seed: int = config.VERIFY_SEED,
    count: int = MESSAGE_COUNT,
    work_scale: float = 1.0,
) -> List[Dict[str, Any]]:
    """Return count related task messages, in the order they will be sent."""
    rng = random.Random(seed)
    per_entity = count // len(ENTITIES)
    order = [entity for entity in ENTITIES for _ in range(per_entity)]
    rng.shuffle(order)
    next_seq: Dict[str, int] = {}
    messages: List[Dict[str, Any]] = []
    for index, entity in enumerate(order):
        next_seq[entity] = next_seq.get(entity, 0) + 1
        seq = next_seq[entity]
        messages.append(
            {
                "correlation_id": f"ord-{run_tag}-{index:02d}",
                "stage": "plan",
                "question": f"Apply update {seq} to trial listing {entity}",
                "plan": [
                    "Read the current listing",
```

Figure 29 - Code: partition key and message generator (hw8/ordering.py)

```
Processing order (unkeyed): finish order -> writer/partition entity#seq
  1. writer A partition 1 CT-2026-108 #1
  2. writer B partition 0 CT-2026-110 #1
  3. writer A partition 1 CT-2026-109 #1
  4. writer B partition 0 CT-2026-106 #1
  5. writer A partition 1 CT-2026-107 #3
  6. writer B partition 0 CT-2026-108 #2
  7. writer B partition 0 CT-2026-107 #1
  8. writer B partition 0 CT-2026-110 #2
  9. writer A partition 1 CT-2026-109 #2
 10. writer B partition 0 CT-2026-107 #2
 11. writer A partition 1 CT-2026-106 #2
 12. writer B partition 0 CT-2026-110 #3
 13. writer A partition 1 CT-2026-106 #3
 14. writer B partition 0 CT-2026-108 #3
 15. writer A partition 1 CT-2026-109 #4
 16. writer B partition 0 CT-2026-107 #4
 17. writer B partition 0 CT-2026-110 #4
 18. writer B partition 0 CT-2026-109 #3
 19. writer B partition 0 CT-2026-108 #4
 20. writer B partition 0 CT-2026-106 #4

Per-entity result (unkeyed):
 CT-2026-106: partitions [0, 1] writers ['A', 'B'] processed seq [1, 2, 3, 4] -> in order
 CT-2026-107: partitions [0, 1] writers ['A', 'B'] processed seq [3, 1, 2, 4] -> OUT OF ORDER
 CT-2026-108: partitions [0, 1] writers ['A', 'B'] processed seq [1, 2, 3, 4] -> in order
 CT-2026-109: partitions [0, 1] writers ['A', 'B'] processed seq [1, 2, 4, 3] -> OUT OF ORDER
 CT-2026-110: partitions [0] writers ['B'] processed seq [1, 2, 3, 4] -> in order
TOTAL (unkeyed): entities out of order=2, entities spread over both partitions=4, order inversions=2
```

Figure 30 - Ordering without a key: out-of-order result table

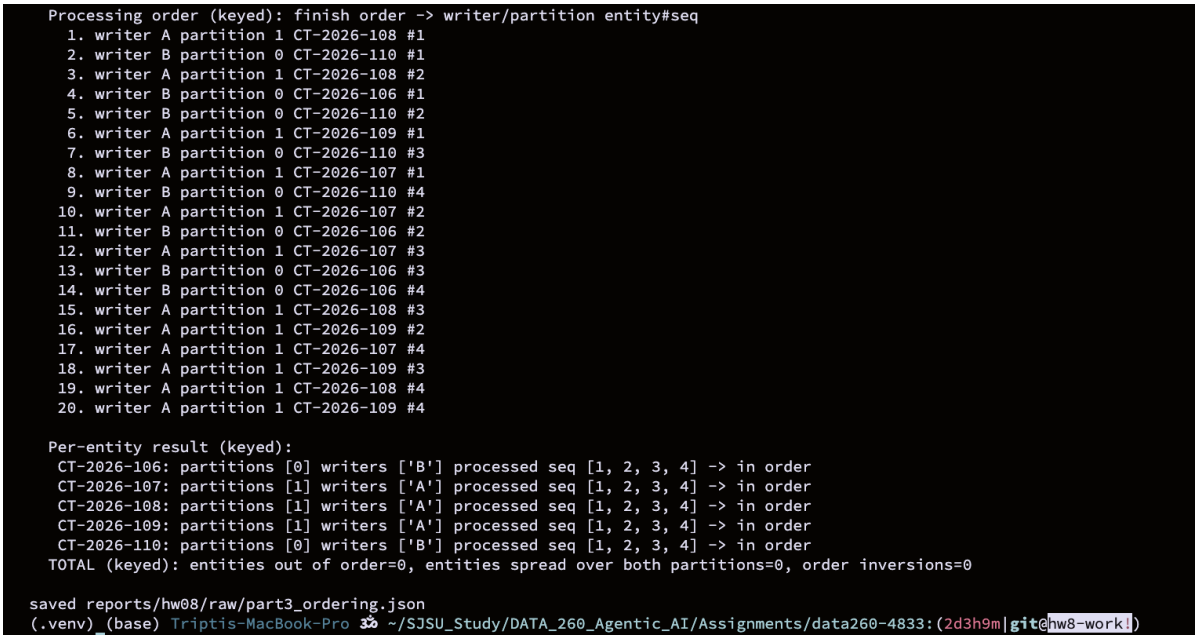


Figure 31 - Ordering with the entity as key: in order

Run	Messages processed	Order inversions	Entities out of order	Entities split over both partitions
No key (random partition)	20	2	2	4
Key = entity	20	0	0	0

The key only keeps order inside one entity. Different entities can still finish in any order, which is fine for this homework because nothing depends on it.

Failure Table

Failure	Before the fix	After the fix	Evidence
Duplicate delivery	Same message sent twice -> 2 drafts (work done twice)	1 draft; replay logged as DUPLICATE and skipped (1 skipped)	Figures 21 and 22
Consumer dies mid-work	Offset committed on receive; after restart redelivered = False, draft produced = False (message lost)	Offset committed after success; redelivered = True, draft produced = True	Figures 24 and 25
Poison message	Retried without limit (14 attempts seen in the window); next message processed = False	3 attempts, then 1 message in s4833_dead_letter; next message processed = True	Figures 27 and 28
Two partitions / two consumers	No key: related messages split over both partitions; 2 order inversions, 2 entities out of order, 4 entities spread over both partitions	Entity id as key: 0 inversions, 0 entities out of order, all messages of an entity stay in one partition	Figures 30 and 31

Comparison with the HW2 In-Process Graph

In HW2 my Planner, Reviewer and Supervisor were nodes of one LangGraph in a single Python process. They shared an AgentState dictionary, called each other as function calls and a turn ceiling in the Supervisor stopped endless loops. In HW8 the agents are separate processes that only share messages.

Two benefits

- Independence and scaling. Each agent can crash, be restarted or be run twice without the others noticing. In Part 3.4 I ran two Writers in one group and Kafka split the partitions between them. In HW2 one exception in a node stopped the whole graph run.
- A durable record. Every hop is stored in a topic with a partition and an offset, so I can replay a message or look at exactly what the Planner sent. In HW2 the state existed only in memory while the run was alive.

Two new responsibilities

- Delivery has to be handled by me. A function call either happens or raises; a message can be delivered twice or lost. I had to add idempotency, commit-after-success and a bounded retry with a dead-letter topic (Parts 3.1 to 3.3) just to get back the safety the graph gave for free.
- Ordering and operations. In the graph the order was the order of the edges. With partitions I must choose a key to get order, and I must also run and watch Kafka itself (topics, partitions, consumer groups, the broker container).

There is also a cost in speed, shown in the JMeter section below. One thing I did not rebuild: the HW2 Supervisor could send a rejected draft back to the Planner. In HW8 the Reviewer only marks the draft approved or rejected; there is no loop back.

JMeter Load Test

I compared the two FastAPI routes with Apache JMeter: /api/hw8/entity (through Kafka and the worker) and /api/hw8/entity-direct (no Kafka). Both plans use 100 threads, a 10 second ramp-up and 1 loop, an HTTP POST with a JSON body (the trial_code contains the thread number so every request creates a different trial) and a Content-Type: application/json header. A response assertion checks for success true. Each plan has the Results Tree, Summary Report, Aggregate Report and Response Time Graph listeners. The numbers below are exported from the listeners (Save Table Data) and summarised by make_hw08_metrics.py. The plan settings are in Figure 32, the Kafka results in Figure 33, the direct results in Figure 34 and the graph in Figure 35.

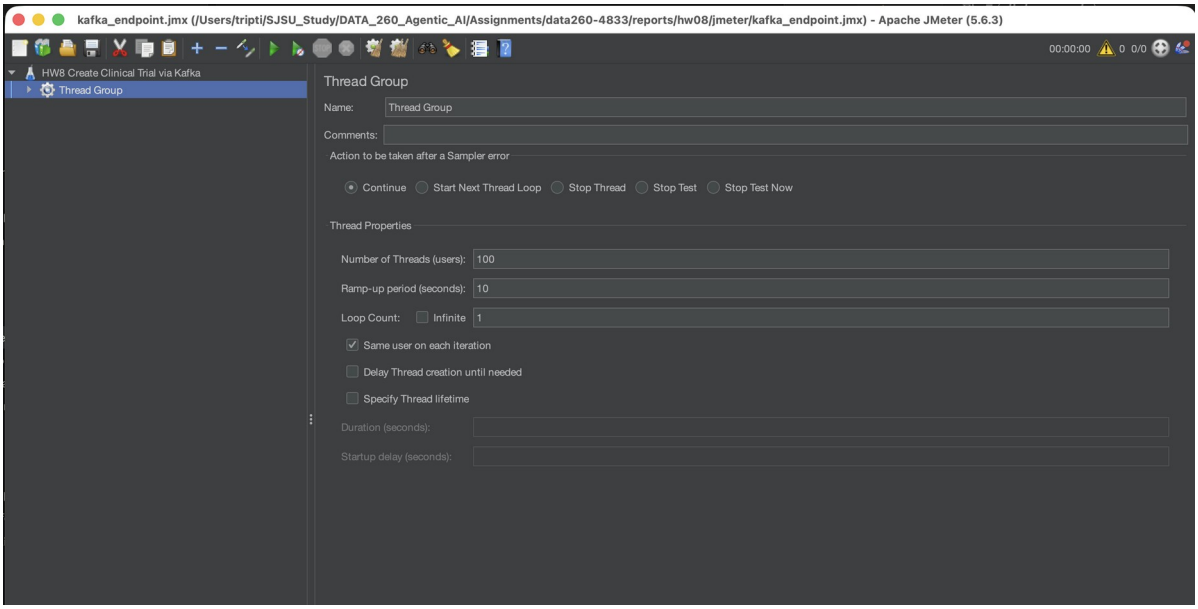


Figure 32 - JMeter plan for the Kafka endpoint (threads, ramp-up, loop)

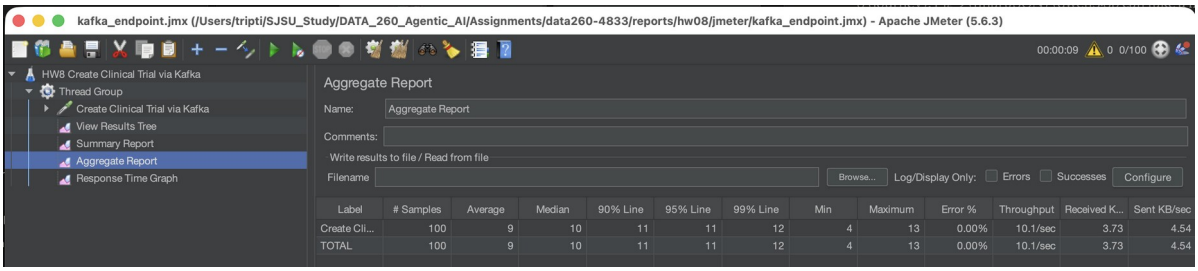


Figure 33 - JMeter Kafka endpoint: Aggregate Report (100 samples, 0% errors)

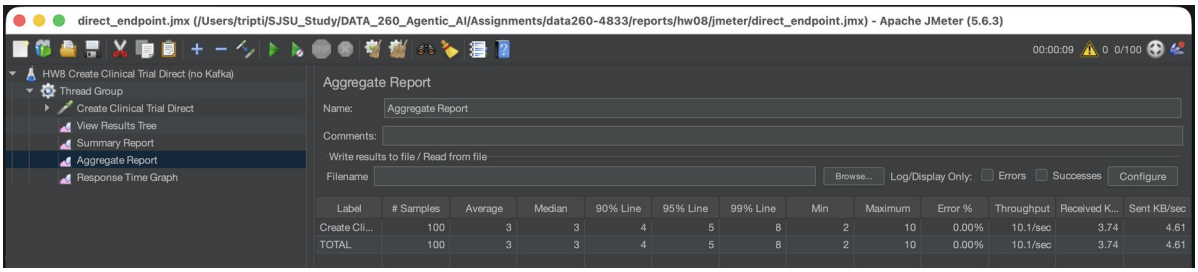


Figure 34 - JMeter direct endpoint: Aggregate Report (100 samples, 0% errors)

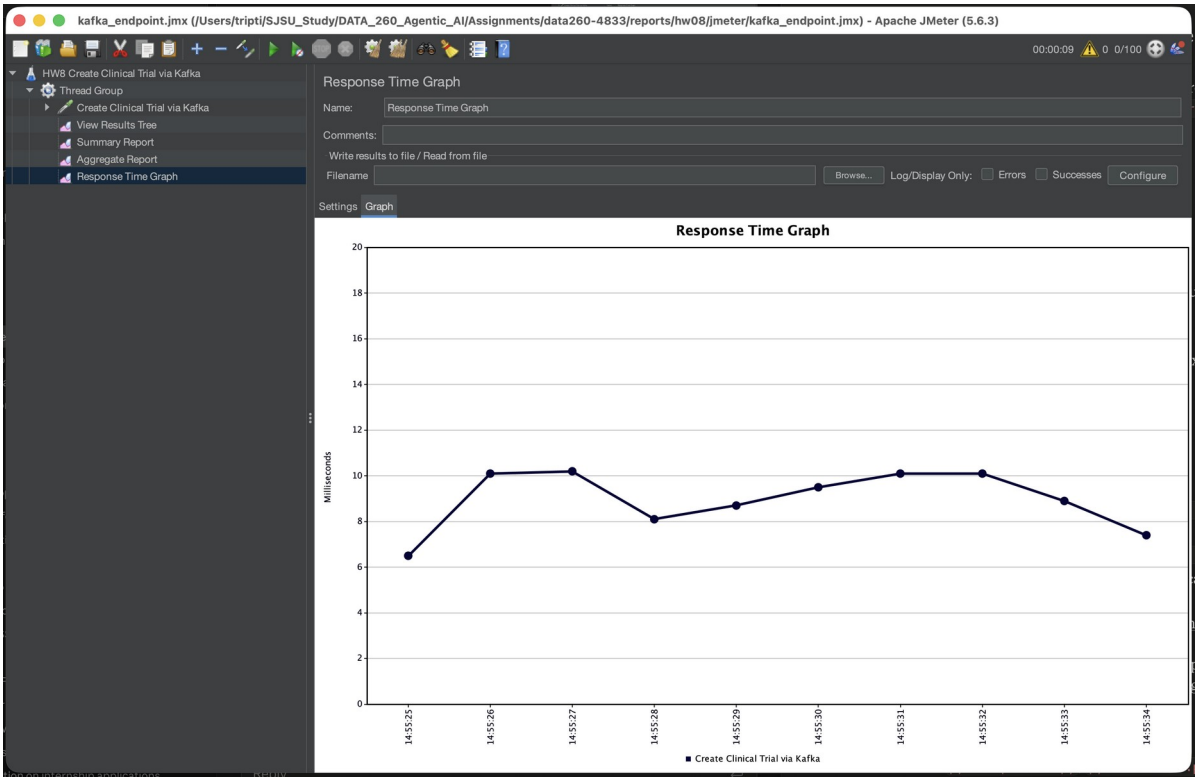


Figure 35 - JMeter Response Time Graph for the Kafka endpoint (1 s intervals)

Endpoint	Samples	Avg (ms)	Min (ms)	Max (ms)	p95 (ms)	Error %	Req/s
Kafka request/response	100	9.0	4.0	13.0	11.0	0.0%	10.09
Direct (no Kafka)	100	3.0	2.0	10.0	5.0	0.0%	10.1

On average the Kafka endpoint took 9.0 ms per request and the direct endpoint 3.0 ms, so the trip through Kafka added about 6 ms per request. Error rates were 0.0% (Kafka) and 0.0% (direct).

Self-check

make verify-hw08 runs verify_hw08.py. It checks the topics, a create-then-get round trip with the correlation_id kept, the error paths, the FastAPI routes, one question through the pipeline, the three failure fixes, the ordering key, the offline tests and that the JMeter plans are present, and writes reports/hw08/verification.json (Figure 36). The Part 3 failure runs (and the checks in this script) use a fixed-text stand-in instead of qwen3:8b for the Writer, because what they test is delivery, offsets and ordering, not the quality of the text, and a stand-in makes the runs short and repeatable. Part 2 uses qwen3:8b.

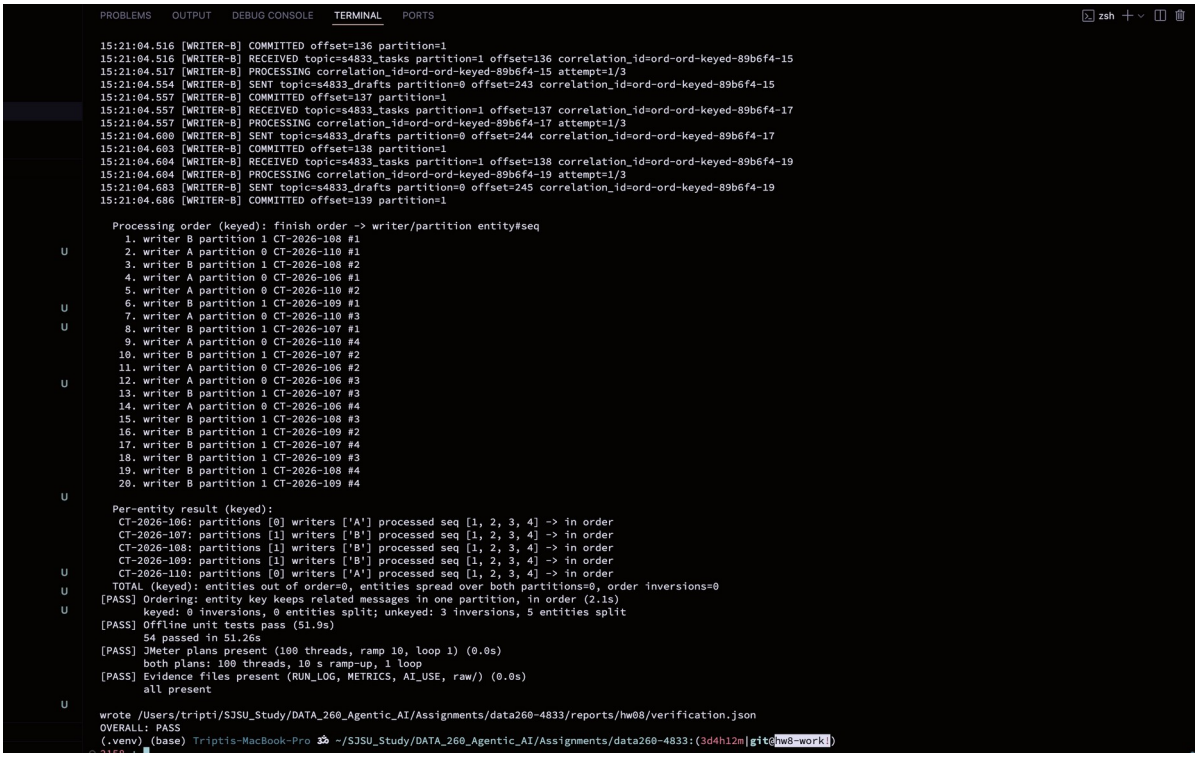


Figure 36 - make verify-hw08: all checks pass

AI Use

1. What I used an AI assistant for, and what I did myself

I used Claude and ChatGpt as a learning tool which helped me understand the assignment & the process of Kafka worker, the FastAPI wrapper, JMeter plans.

I started Kafka with Docker, ran every command on my own Mac, ran the model (qwen3:8b through Ollama) and ran JMeter myself, so every number in the report comes from my runs. I took all the screenshots and made the report. I read each log and result before it went into the report. For example, I reran the ordering test when the first run came out in order by luck, and I cleared JMeter and ran it again when the sample count was wrong.

2. One AI output that was wrong

In Part 3 the poison-message test printed a summary line that said 14 failed attempts. But when I counted the FAILED lines in the Writer log from the same run, there were 18. The summary looked fine on its own, and the "after the fix" case (3 attempts, then the dead-letter topic) was correct, so the mismatch was easy to miss. The summary was wrong because the test counted the attempts while the Writer was still retrying in the background.

A smaller case: my first unkeyed ordering run came out completely in order, with zero inversions. That looked like the order problem did not exist, but it only meant the two Writers happened to be fast enough that run. A third case was in JMeter, where the Summary Report showed 200 and then 300 samples for a plan with 100 threads and loop count 1.

3. How I detected it

For the poison test I compared the summary line with the real Writer log of the same run. The numbers did not match (14 against 18), so I knew the test, and not Kafka, was wrong.

For ordering, I did not trust a result that showed nothing, so I ran the test again. The second run split four entities across both partitions and gave two inversions (CT-2026-107 as 3, 1, 2, 4 and CT-2026-109 as 1, 2, 4, 3).

For JMeter, 100 threads with one loop must give exactly 100 samples, so 200 and 300 told me the old results were still in the listeners.

4. What changed and why it works now

The poison test now stops the Writer first and only then counts the attempts, so the summary and the log agree. After the fix my rerun showed 14 in both places. I reran the ordering test, and the report text describes the result that actually happened. The unkeyed result changes from run to run (I saw between 2 and 5 inversions), but the keyed run was always 0 inversions, because the entity id as the key keeps one entity in one partition. In JMeter I press Clear All before every run. Both plans now show exactly 100 samples with 0.00% errors.

I note two limits in the report. The failure tests use a stub Writer so that they are fast and repeatable, and the unkeyed ordering numbers depend on timing, so I report them as examples and not as fixed values.